C# Övning 6 – Garage 2.0

Ett första övergripande projekt

För att koppla samman mycket av det ni lärt er ska vi nu bygga en garage-/konsolapplikation. Applikationen ska tillhandahålla den funktionalitet ett system kan behöva för att simulera ett enkelt garage.

Det ska alltså gå att parkera fordon, hämta ut fordon, se vilka fordon som finns i garaget och vilka egenskaper de har. Allt detta ska ske i en konsolapplikation med huvudmeny och undermenyer.

Anledningen till att ni ska programmera ett garage är att det är enkelt att förankra uppdelningen i systemet. Vi kan huvudsakligen dela upp ett garage i följande delar:

- **Garaget:** En representation av själva byggnaden. Garaget är en plats där ett antal fordon kan förvaras. Garaget kan därför representeras som en samling av fordon.
- **Fordon:** Bilar, motorcyklar, enhjulingar eller andra typer av fordon som kan ställas in i garaget.

Detta är de två huvudsakliga "objekttyperna" i ett fysiskt garage. Om vi tittar närmare bör det också finnas subklasser till fordon, alltså att varje fordonstyp är en egen subklass i systemet. Utöver detta krävs funktionalitet som hanterar att fordon ställs in i garaget, att fordon kan tas ut ur garaget samt att vi kan presentera och söka bland fordonen.

I mer programmeringsvänliga termer ska vi som minimum ha:

- En kollektion av fordon; klassen **Garage**.
- En fordonsklass; klassen **Vehicle**.
- Ett antal subklasser till fordon.
- Ett användargränssnitt som låter användaren använda garagefunktionerna. Här sker all interaktion med användaren.

Kravspecifikation

Fordonen ska implementeras som klassen **Vehicle** och subklasser till den.

- **Vehicle** innehåller samtliga egenskaper som ska finnas i samtliga fordonstyper, till exempel registreringsnummer, färg, antal hjul och andra egenskaper ni kan komma på.
- Registreringsnumret ska vara unikt.

- Det måste minst finnas följande subklasser:
    - **Motorcycle**
    - **Airplane**
    - **Car**
    - **Bus**
    - **Boat**

Dessa subklasser ska implementera minst en egen egenskap var, till exempel:

- Number of Engines
- Cylinder volume
- Fuel type (Gasoline/Diesel)
- Number of seats
- Length

Klassen behöver inte ärva från någon annan klass eller implementera något interface.

Samlingen av fordon ska internt i klassen **Garage** hanteras som en **List<Vehicle>**. Listan ska vara privat. Vid instansieringen av ett nytt garage måste kapaciteten anges som argument till konstruktorn.

VI SKA ANVÄNDA OSS AV EN List<Vehicle> INTERNT I GARAGE-KLASSEN!

Ni ska **inte** använda:

- En array internt i Garage-klassen.
- Generics, exempelvis Garage<T> where T : Vehicle.
- LINQ.
- Enhetstester.

Funktionalitet

Det ska gå att:

- Lista samtliga parkerade fordon.
- Lista fordonstyper och hur många av varje typ som står i garaget.
- Lägga till och ta bort fordon ur garaget.
- Sätta en kapacitet, alltså antal parkeringsplatser, vid instansieringen av ett nytt garage.

- Möjlighet att populera garaget med ett antal fordon från start.
- Hitta ett specifikt fordon via registreringsnumret. Sökningen ska fungera oavsett om användaren skriver ABC123, Abc123 eller AbC123.
- Söka efter fordon utifrån en eller flera egenskaper från basklassen **Vehicle**. Exempel:
    - Alla svarta fordon med fyra hjul.
    - Alla motorcyklar som är rosa och har tre hjul.
    - Alla lastbilar.
    - Alla röda fordon.

Användaren ska få tydlig feedback på om något gått bra eller dåligt. Till exempel ska användaren få bekräftelse när ett fordon har parkerats. Om det inte går ska användaren få veta varför.

Användargränssnitt

Programmet ska vara en konsolapplikation med ett textbaserat användargränssnitt.

Från gränssnittet ska det gå att:

- Navigera till all funktionalitet i garaget.
- Skapa ett garage med en användarspecificerad storlek.
- Stänga av applikationen.

Applikationen ska hantera felaktiga indata på ett robust sätt, så att den inte kraschar vid felaktig inmatning eller användning.

Förslag på extra funktionalitet

Följande funktionalitet är inte ett krav, men kan läggas till för att utöka projektet:

- Möjlighet att även söka på de fordonsspecifika egenskaperna.
- Hantera flera garage med olika typer av fordon, till exempel en hangar, ett vanligt garage och ett motorcykelgarage.
- Möjlighet att växla mellan olika garage i användargränssnittet, så att tidigare nämnda funktioner bara utförs på det garage som för tillfället är valt.
- Tydlig visning av vilket garage användaren för närvarande arbetar med.
- Ett garage som består av parkeringsplatser, där varje parkeringsplats kan hålla ett fordon.

- Spara och läsa garage till och från filsystemet, exempelvis via meny eller automatiskt vid start och avstängning.
- Olika fordon tar olika mycket plats. Till exempel kan en bil ta en plats, en båt två platser, ett flygplan tre platser och en motorcykel en tredjedels plats.
- När man parkerar ska endast de fordon som garaget har plats för visas som alternativ.
- Läsa in garagets storlek via konfiguration.
- Valfri ytterligare funktionalitet som ni tycker borde finnas.

Lycka till!